

BASP: Communication-Efficient Batch-Aware Sequence Parallelism for LLM Training

Bigyan Ghimire, Jon C. Calhoun

*Holcombe Department of Electrical and Computer Engineering
Clemson University
Clemson, USA*

Abstract—Long-context reasoning for large language models (LLMs) is becoming increasingly important, but training over long sequences remains challenging due to massive memory and communication requirements. Sequence parallelism has emerged as an essential technique for addressing bottlenecks in long sequence LLM training. However, we observe that existing sequence parallelism methods are batch-agnostic and apply uniform sequence partitioning across all batch sizes, resulting in inefficient communication. In this paper, we introduce Batch-Aware Sequence Parallelism (BASP), a sequence parallelism approach that leverages batch structure to reduce communication overhead. BASP exploits batch structure by partitioning GPUs into disjoint sequence-parallel groups according to the micro-batch size. This design reduces the all-to-all communication group size, thereby localizing communication and improving training efficiency. Experimental results on an NVIDIA A100 cluster show that BASP improves end-to-end training time by up to $1.17 - 1.31\times$ in Llama and Qwen models compared to standard sequence parallel baselines, while preserving identical model accuracy and memory usage.

Index Terms—DeepSpeed-Ulysses, sequence parallelism, all-to-all

I. INTRODUCTION

Transformer-based large language models (LLMs) have gained wide attention as they have demonstrated exceptional capabilities in long context tasks such as long dialogue processing [5], large document analysis [7], processing complex codebases [22], and audio, video, and image processing with multi-modal models [28]. For instance, when summarizing a book or analyzing a large document, LLMs require long context reasoning over thousands to millions of tokens in a single prompt. While early transformer models operated on a sequence length of 512–2048 tokens [18], current transformer models operate at context windows exceeding 100K tokens [3] [1]. For example, ChatGPT supports 128K tokens [20], while Claude models provide a context window of up to 1M tokens [1]. This has surged the demand for training LLMs in long context windows.

Distributed training strategies such as data parallelism (DP), pipeline parallelism (PP), and tensor parallelism (TP) do not address the long-sequence memory-scaling bottleneck in the transformer training. Data parallelism, which replicates the model across GPUs and partitions the batch, suffers from per-GPU memory pressure [11]. Pipeline parallelism partitions layers across devices, reducing per-GPU memory pressure but introducing pipeline bubbles, which cause significant under-

utilization of GPUs [12] [2]. Tensor parallelism partitions individual weight matrices across GPUs [23], resulting in multiple GPUs executing a general matrix multiplication (GEMM). However, it does not parallelize the sequence dimension, which is the main memory bottleneck for long context training [13] [15].

Recently, for long-context training, sequence parallelism (SP) has been studied as a promising approach [13] [9] [14] [16]. SP partitions the sequence dimension across GPUs, distributing both the sequence length and the corresponding activation memory proportionally to the number of participating GPUs. Megatron Sequence Parallelism (Megatron-SP) [14] extends Megatron-LM tensor parallelism by additionally parallelizing the non-tensor parallel layers like layernorm and dropout layers across the sequence dimension [14]. However, the communication volume in Megatron-SP increases linearly with message size, regardless of the number of GPUs [13]. SP introduced by DeepSpeed-Ulysses (Ulysses-SP) [13], provides a more scalable approach to long context training. Unlike Megatron-SP, which applies sequence partitioning for some layers, Ulysses-SP partitions the sequence dimension for all the layers, including attention and Multi Layer Perceptron (MLP). For attention computation, it employs an all-to-all communication to distribute sequence partitions across the GPUs and computes attention for different heads in parallel. Then it employs another all-to-all to redistribute the attention output for the sequence partitions.

However, as the model scale grows, the training efficiency becomes increasingly constrained by communication rather than computation [6] [10]. Different parallelism strategies exhibit fundamentally different communication patterns. While existing studies [17] [19] [27] have aimed to mitigate communication overhead in traditional parallelism, communication inefficiency in sequence parallelism has been understudied.

In standard Ulysses, all N GPUs must communicate with $N - 1$ peers for all-to-all to redistribute the entire batch¹ of sequences during attention computation. Our profiling on an 8-GPU NVIDIA A100 cluster (2 nodes $\times$ 4 GPUs/node) reveals that this collective becomes a critical bottleneck: as batch size B increases, all-to-all time grows proportionally, consuming an increasing fraction of total iteration time (Figure 1).

¹In this paper, we use batch for convenience, to mean micro-batch, which is the per-GPU fraction of the global batch.

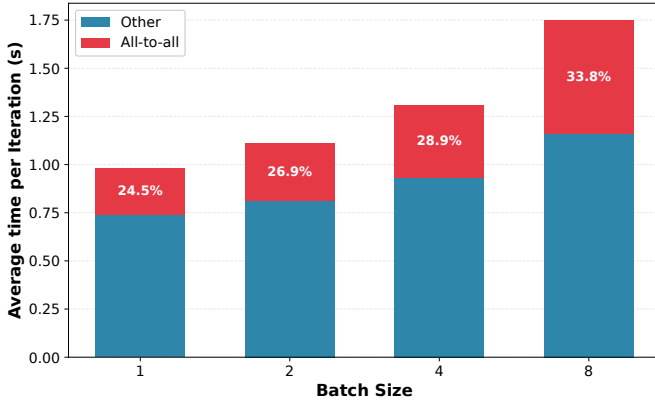


Fig. 1: All-to-all’s share of time increases linearly with micro-batch size (sequence length = $8K$, $N = 8$, Llama-3.2-1B)

Key insights and contributions. We observe that standard Ulysses-SP performs global N -way all-to-all regardless of micro-batch size B during attention computation. However, when $B > 1$ and N is divisible by B (i.e., $N = KB$), this global communication is unnecessary. We instead assign each of the B sequences to a disjoint group of $K = N/B$ GPUs, decomposing the single N -way collective into B independent K -way collectives executing in parallel. This reduces the number of communication peers per GPU from $N-1$ to $K-1$, with no increase in per-GPU memory footprint or sequence shard size.

Crucially, on typical clusters where intra-node bandwidth (NVLink) significantly exceeds inter-node bandwidth (InfiniBand), setting K equal to GPUs per node confines all-to-all traffic to the faster intra-node network.

With these observations, this paper makes the following contributions:

- Based on our observations, we propose *Batch-Aware Sequence Parallelism* (BASP), where a subgroup of GPUs handles a subset of the batch. This replaces global all-to-all communication with smaller intra-group collectives while keeping the per-GPU memory footprint and sequence shard size unchanged.
- We evaluate our approach on variants of Llama and Qwen model families. Our results show that Llama 3.1-8B achieves a speedup of $1.21\times$ and Qwen 1.5-1.8B achieves a speedup of $1.32\times$ compared to Ulysses-SP. Our results also show a reduction of all-to-all time by up to $85\times$ in micro-batch size of 8.

The remainder of this paper is organized as follows. Section II provides background on transformers and parallelism methods. Section III provides the observation and motivation for our work. Section IV details the design and implementation of BASP. Section V presents experimental results and analysis. Section VI discusses the limitations, Section VII discusses the related works, and Section VIII concludes.

II. BACKGROUND

A. Transformer Architecture

The transformer architecture [24] is the foundation of the large language models. At its core, the transformer processes input sequences through stacked layers of self-attention and feed-forward networks. The self-attention mechanism enables each token to attend to all other tokens in the sequence, capturing long-range dependencies that are crucial for language understanding.

As model and sequence lengths scale, the quadratic complexity $O(N^2)$ of attention computation becomes the primary bottleneck [8]. This has motivated various parallelization strategies to distribute attention computation across multiple GPUs. To understand these strategies, we first review the multi-head attention mechanism that underlies all modern transformers. The multi-head attention is based on the scaled dot-product attention.

Scaled Dot-Product Attention: Given L tokens $X \in \mathbb{R}^{L \times d_{in}}$ and L' tokens $X' \in \mathbb{R}^{L' \times d_{in}}$ of dimensions d_{in} dimensions each, the scaled dot-product attention is computed as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

with $Q = XW_Q$, $K = X'W_K$, $V = X'W_V$

where $W_Q \in \mathbb{R}^{d_{in} \times d_k}$, $W_K \in \mathbb{R}^{d_{in} \times d_k}$ and $W_V \in \mathbb{R}^{d_{in} \times d_v}$ are the projection matrices for the query, key and values respectively. In self-attention, the input tokens are the same, i.e., $X = X'$.

Multi-Head Attention: To capture different aspects of the relationship in the tokens, multi-head attention was introduced [24]. If h is the number of heads, multi-head attention calculates h different low-dimensional projections of the matrices (Q, K, V) , performs scaled-dot product attention for each head, concatenates the outputs, and applies a concatenation to the projection output.

$$\text{Multihead}(X, X') = \text{Concat}(H_1, H_2, \dots, H_h)W_O$$

with $H_i = \text{Attention}(XW_Q^i, X'W_K^i, X'W_V^i)$

where $W_Q^i \in \mathbb{R}^{d_{in} \times d_k/h}$, $W_K^i \in \mathbb{R}^{d_{in} \times d_k/h}$ and $W_V^i \in \mathbb{R}^{d_{in} \times d_v/h}$ are the i_{th} projection matrices for the query, key and values respectively. $W_O \in \mathbb{R}^{d_{dv} \times d_{out}}$ is the final output projection matrix.

Memory bottleneck: The attention computation in transformer models is a major computational and memory bottleneck due to its quadratic complexity with respect to the sequence length. In particular, the attention score matrix QK^T requires $O(N^2)$ memory, which poses a memory constraint for training.

This challenge is especially amplified in long-context distributed training for large language models, where activation memory grows rapidly with increasing sequence length. Even when model parameters are distributed across multiple GPUs,

the intermediate attention activations may exceed the memory capacity of a single device.

Techniques like FlashAttention [8] reduce memory overhead by avoiding explicit materialization of the full attention matrix by using tiling and online softmax attention, reducing memory complexity from $O(N^2)$ to $O(N)$ for intermediate activations. However, despite this improvement, long-context training still encounters memory and communication bottlenecks at scale [26], motivating sequence-parallel approaches such as DeepSpeed-Ulysses.

B. Modes of parallelism

Training large language models at scale relies on multiple forms of parallelism, each targeting a different computational or memory bottleneck.

Data Parallelism (DP): In data parallelism, the input batch is partitioned across participating GPUs, with each GPU maintaining a full replica of the model parameters. Each device performs forward and backward computation on its local mini-batch, followed by gradient synchronization across all devices. Due to its replication of model parameters (gradients, optimizer states, and weights) across all devices, training large models is not feasible because a single device cannot hold the whole model.

ZeRO / Fully Sharded Data Parallel: ZeRO [21] and Fully Sharded Data Parallel (FSDP) [30] improve upon standard data parallelism by partitioning model states across devices instead of fully replicating them on each GPU. In ZeRO, optimizer states (ZeRO-1), gradients (ZeRO-2), and model weights (ZeRO-3) are sharded instead of fully replicated. FSDP follows a similar strategy and is functionally closest to ZeRO-3. This type of partitioning reduces the per-GPU memory usage and allows training models that exceed the memory capacity of a single device.

Tensor Parallelism (TP): Tensor parallelism, introduced by Megatron-LM [23], partitions the computation within individual layers across multiple GPUs. Commonly, large matrix multiplications in MLP and attention layers are split along the hidden or output dimensions, allowing each GPU to compute a portion of the GEMM operation. In order to reconstruct the full activation, GPUs must exchange intermediate activations using collective communication operations such as all-reduce, all-gather, or reduce-scatter. For attention computation, Megatron-LM partitions the sequence along sequence dimensions and applies all-gather and reduce-scatter collective to aggregate QKV projections. While TP reduces memory and compute pressure per device, these communication operations introduce overhead.

Pipeline Parallelism (PP): Pipeline parallelism (PP) involves partitioning the layers into subgroups of layers known as stages and assigning the stages to different GPUs. Micro-batches are then pipelined through these stages in the forward and backward passes. PP is effective at alleviating the huge memory pressure on a GPU for very deep transformer models, but it introduces pipeline bubbles and scheduling complexity,

which lead to underutilization of GPUs and therefore reduced training efficiency.

Sequence Parallelism (SP): Megatron-SP [14] adds parallelization to layers that were previously not tensor-parallelized in Megatron-LM. The input is partitioned in the sequence dimension for operations such as layer normalization and dropout, while tensor parallelism is still used for linear layers, including attention and MLP. Communication volume in Megatron-SP’s sequence parallelism increases linearly with the sequence length, irrespective of the number of accelerators.

Context Parallelism (CP): Context parallelism (CP) was introduced by DeepSpeed-Ulysses [13] to support extremely long context lengths. Unlike standard sequence parallelism, CP distributes the full sequence across devices for attention computation itself, allowing training on context windows that would otherwise exceed device memory limits. It maintains a constant communication volume regardless of the increase in sequence lengths and device counts, achieving increased training efficiency.

C. DeepSpeed Ulysses

DeepSpeed-Ulysses partitions the individual sequence along the sequence dimension among participating GPUs. For micro batch size B , sequence length S , and number of GPUs N , each GPU holds $\frac{B \cdot S}{N}$ tokens. Then, each GPU calculates the Q, K, V matrices for their portion of the sequence tokens independently for all heads. Then, an all-to-all between $S_p = N$ (degree of parallelism) GPUs is initiated, which distributes the Q, K, V projections across various GPUs that are responsible for processing different heads for the full sequence. This allows the heads to be computed in parallel, independently. Then another all-to-all is initiated to redistribute attention output of the tokens back to the GPUs that held the sequence partitions initially, so other layers like MLP, LayerNorm, etc. continue. An example is presented in Figure 2a.

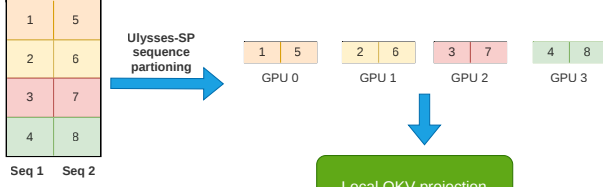
This design allows attention computation to remain mathematically equivalent to full self-attention while reducing per-device memory usage through sequence partitioning, at the cost of an additional all-to-all communication step.

III. OBSERVATION AND MOTIVATION

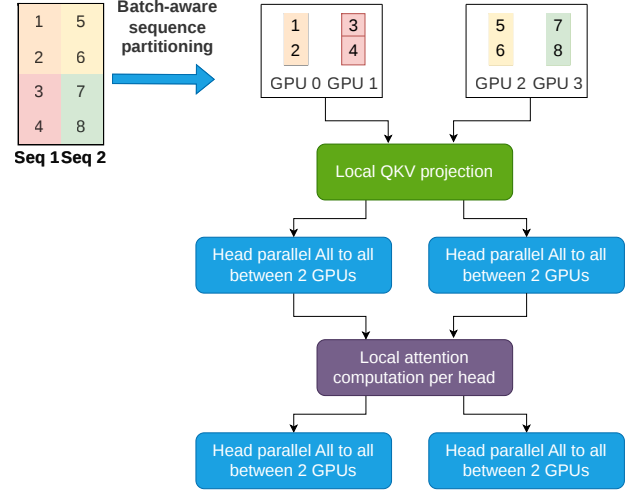
A. Ulysses-SP Communication pattern

In Ulysses-SP, each sequence in a micro-batch is partitioned among N GPUs (S_p degree), such that each GPU holds a chunk of tokens from all sequences in the micro-batch. During the attention computation, N GPUs participate in the all-to-all for head-parallel attention computation. This incurs N -way all-to-all across GPUs.

We profile the GPU kernels during a single iteration of sequence parallel training (Figure 5). Profiling shows that all-to-all accounts for a non-negligible portion of the execution time, i.e., 34%. Figure 1, shows that the all-to-all communication time increases as micro-batch size increases. In a network-constrained cluster, the cost of transmitting these data crossing the entire cluster cannot be easily hidden via pipelining the communication and computation.



(a) Deepspeed Ulysses workflow.



(b) Batch-aware Sequence Parallelism (BASP) workflow reducing all-to-all.

Fig. 2: An example showing the working of Ulysses-SP and BASP in high-level for a micro-batch size of 2 sequences and 4 GPUs. The sequence is sharded among the GPUs equally. The methods differ in the sequence partitioning pattern. Each GPU then calculates the Q, K, and V matrices for their portion of tokens for all heads. Then, in Ulysses-SP, a single all-to-all happens between 4 GPUs, while in BASP, two all-to-alls happen between 2 GPUs in parallel for attention calculation. After the attention calculation, the attention output is redistributed back to the original GPUs using another all-to-all.

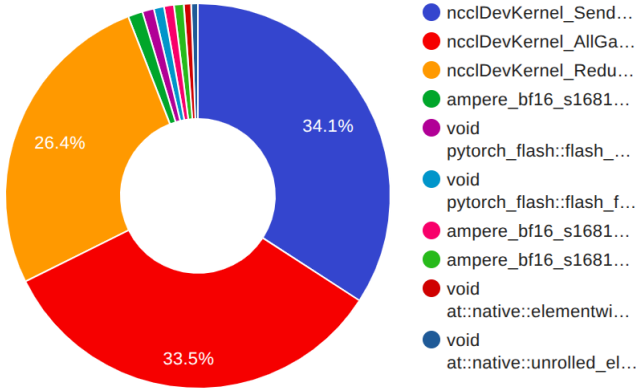


Fig. 3: Percentage of time spent during GPU kernel execution on an iteration of SP training (Llama-3.2-3B). NcclSend_Recv(blue) (used to implement all-to-all in NCCL) shows 34% of total execution time.

Motivation: Our core idea is to exploit the micro-batch size to reduce the number of participants involved in each all-to-all communication, thereby improving training efficiency. This intuition is consistent with classical collective communication models such as the α - β model, where communication cost

consists of a latency term and a bandwidth term.² This motivates us to design a new system that reduces the cost of all-to-all communication while preserving the generality and usability advantages. By partitioning communication into smaller groups determined by the micro-batch structure, our approach limits synchronization to fewer participants. Our method jointly exploits both the batch and sequence dimensions to reduce the number of communicating devices in a group during attention computation, which allows faster all-to-all communication. This decreases latency overhead and improves communication efficiency, while still preserving the correctness of sequence parallel execution. We give detailed descriptions of our methodology in the next section.

IV. BATCH AWARE SEQUENCE PARALLELISM

Overview: BASP exploits micro-batch structure to reduce the all-to-all overhead. Ulysses-SP performs a global N -way all-to-all across all GPUs to redistribute sequence chunks during attention computation, regardless of batch size. When micro-batch size B satisfies $B > 1$ and $N = KB$ (where K is an integer), we can exploit this structure to reduce communication overhead. Instead of distributing all B sequences among all N GPUs, we assign each sequence to a disjoint group of $K = N/B$ GPUs. Each group performs an independent K -way all-to-all on its assigned sequence, reducing per-GPU communication peers from $N - 1$ to $K - 1$

²Under the standard α - β communication model, a pairwise-exchange all-to-all can be approximated as $T_{a2a} = (P - 1)(\alpha + m\beta)$, where P is the number of participating ranks, m is the message size exchanged per round, α is the latency, and β is the inverse bandwidth.

during the attention computation. When K equals the number of GPUs per node, all communication remains on fast intra-node interconnects (NVLink), avoiding slower inter-node links (InfiniBand) entirely.

Our method spans three modules: (1) Batch-aware group formation, (2) Batch-aware Sequence Partitioning, and (3) Subgroup all-to-all.

A. Batch-aware group formation

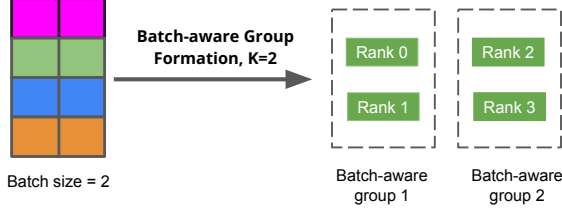


Fig. 4: Batch-aware group formation example in $N = 4$ GPUs with $B = 2$.

We first create multiple batch-aware groups that process the sequences in parallel. While Ulysses-SP creates a single global process group corresponding to the SP degree, BASP partitions the available GPUs into multiple independent subgroups within the SP degree (which we refer to as batch-aware group) at initialization time, with each subgroup responsible for processing a distinct subset of the batch.

Given total world size N and microbatch size B we compute the group size as:

$$K = \frac{N}{B}$$

This formulation ensures that each of the B sequences in the batch is assigned to exactly K GPUs, with no GPU participating in multiple groups (Figure 4). When $B > N$, there are more sequences than GPUs available for partitioning; and thus BASP falls back to standard data parallelism. Currently, BASP assumes that $K = N/B$ is an integer. Extending BASP to support non-divisible configurations is part of our future work. Algorithm 1 shows the complete group formation logic.

On multi-node clusters, communication performance depends critically on whether collectives stay within fast intra-node links (NVLink) or must cross slower inter-node network (InfiniBand). To exploit this hierarchy, we must ensure that each group’s all-to-all traffic remains entirely within a single node when possible.

We achieve this through contiguous rank assignment: GPUs 0 through $K - 1$ form group 0, GPUs K through $2K - 1$ form group 1, and so forth. When K equals the number of GPUs per node, this contiguous assignment guarantees each group maps to exactly one physical node, confining all communication to NVLink when possible.

Algorithm 1 Batch-Aware Process Group Creation

Input: world_size N , batch_size B

Output: Process group for current rank

```

1:  $K \leftarrow N/B$  { // Group size }
2:  $num\_groups \leftarrow B$  { // Number of groups }
3: assert  $N \bmod B = 0$  { // Validate divisibility }
4:
5:  $global\_rank \leftarrow dist.get\_rank()$ 
6: for  $i = 0$  to  $num\_groups - 1$  do
7:    $ranks \leftarrow [i \cdot K, i \cdot K + 1, \dots, i \cdot K + K - 1]$ 
8:    $group \leftarrow dist.new\_group(ranks)$ 
9:   if  $global\_rank \in ranks$  then
10:    return  $group$  { // This rank’s subgroup }
11:   end if
12: end for
```

Algorithm 2 Batch-Aware Sequence Partitioning

Input: Batch $X \in \mathbb{R}^{B \times L \times H}$, group_size K , global_rank r

Output: Local shard X_{local}

```

1:  $my\_group \leftarrow \lfloor rank/K \rfloor$ 
2:  $my\_local\_rank \leftarrow rank \bmod K$ 
3:
4:  $seq\_chunk\_len \leftarrow L/K$ 
5:
6: { // Extract assigned sequence and shard }
7:  $seq\_start \leftarrow my\_group$ 
8:  $seq\_end \leftarrow my\_group + 1$ 
9:  $token\_start \leftarrow my\_local\_rank \cdot seq\_chunk\_len$ 
10:  $token\_end \leftarrow (my\_local\_rank + 1) \cdot seq\_chunk\_len$ 
11:
12:  $X_{local} \leftarrow X[seq\_start : seq\_end, token\_start : token\_end, :]$ 
13: return  $X_{local}$ 
```

B. Batch Aware Sequence Partitioning

The sequence partitioning logic determines how sequences are distributed across GPUs. In Ulysses-SP, for a microbatch of size B and SP degree N , each sequence is partitioned into N contiguous chunks along the sequence dimension, and every GPU processes the corresponding chunk for all sequences in the batch. In contrast, our method jointly exploits both the batch and sequence dimensions. Instead of assigning all GPUs to shard every sequence, we partition the available GPUs into multiple SP groups based on the microbatch size. Then each sequence is only distributed across the GPUs within its assigned group.

This layout preserves the per-GPU computational load while eliminating the need for cross-sequence aggregation inside the SP group. As a result, communication becomes localized to the GPUs assigned to each batch element, improving scalability for long-context training.

Process group assignment. To enable each GPU to determine (1) which sequence(s) it is responsible for, and (2) which shard of that sequence it should process, each rank

is assigned to group $g = \lfloor \text{rank}/K \rfloor$ with local position $\ell = \text{rank} \bmod K$, where K is the group size.

Algorithm 2 shows the complete batch sharding procedure executed during data loading.

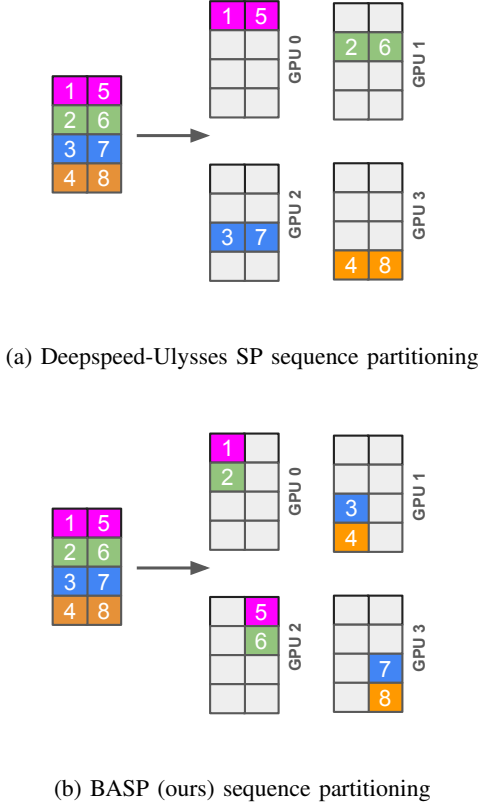


Fig. 5: Example showing a) Ulysses-SP and b) BASP (ours) sequence partitioning for $N = 4$ GPUs and microbatch size $B = 2$, with each sequence of length S . With b), we end up assigning a complete sequence to fewer GPUs, which results in less number of participants in all-to-all during self-attention.

C. Subgroup all-to-all

The attention computation requires all-to-all communication to redistribute QKV projections across attention heads. In Ulysses-SP, this all-to-all spans all N GPUs globally. In BASP, each all-to-all is restricted to the K GPUs within a subgroup.

Our data partitioning, while being computationally similar, reduces the all-to-all communication. With our approach, as shown in Figure 6, instead of performing a single N -way all-to-all across all GPUs, we perform multiple independent K -way all-to-alls within GPU groups ($K = N/B$). Each group processes a subset of the batch, with tokens distributed among the K GPUs in that group.

1) *Workload and Memory Equivalence.*: In BASP, while a GPU ends up processing greater number attention heads than Ulysses-SP, the computational workload and memory remains

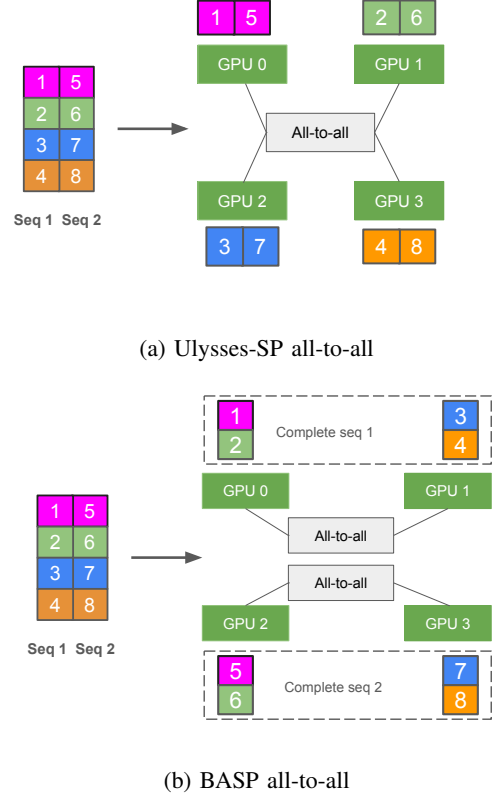


Fig. 6: Example showing a) Ulysses-SP and b) BASP (ours) all-to-all for $N = 4$ GPUs and microbatch size $B = 2$, for seq1 and seq2 . With BASP, 2 GPUs end up holding each complete sequence, resulting in 2 parallel all-to-all between 2 GPUs compared to a single all-to-all between 4 GPUs in Ulysses-SP.

same. Ulysses-SP partitions S across P GPUs, giving per-GPU work:

$$W_{\text{Ulysses}} \propto B \cdot H \cdot \frac{S}{P} \cdot d.$$

In BASP, the P GPUs are divided into G groups. Each GPU processes B/G batch elements and a sequence shard of size $S/(P/G) = SG/P$, yielding:

$$W_{\text{BASP}} \propto \frac{B}{G} \cdot H \cdot \frac{SG}{P} \cdot d = B \cdot H \cdot \frac{S}{P} \cdot d.$$

Thus, BASP preserves per-GPU workload and the same holds for memory.

D. Combining with ZeRO

BASP further integrates with ZeRO [21] to further reduce memory consumption in training large language models. ZeRO eliminates memory redundancy by partitioning the model's optimizer states (Zero-1), gradients (Zero-2), and parameters (Zero-3) across data parallel processes. In BASP, ZeRO partitions model states across both sequence and data parallel groups.

TABLE I: Comparison of Parallelism Strategies (for N GPUs, microbatch size B and sequence length S)

Method	Tokens/GPU	All-to-All Participants
Standard Ulysses ($SP = N$)	BS/N	N GPUs
Standard Ulysses ($SP = N/B$)	B^2S/N	N/B GPUs
BASP (Ours) $SP = N, K = N/B$	BS/N	N/B GPUs

E. Comparison with Sequence Parallel degree

While the sequence parallelism degree in Ulysses-SP, can be set to N/B , setting $S_p = N/B$ does not replicate BASP. Table I shows the difference in our methods. We see that BASP achieves the benefit of both methods: $S_p = N$ and $S_p = N/B$. The key distinction lies in how sequence length and batch dimensions are distributed across GPUs.

Ulysses-SP with $SP = N$ distributes the sequence length evenly across all N with each GPU processing BS/N tokens. To perform attention, all N GPUs participate in a global all-to-all exchange.

Ulysses-SP provides a `sequence_parallel_size` parameter that allows setting $S_p < N$, partitioning GPUs into groups of size N/B . However, this configuration fundamentally changes the workload distribution: each GPU now processes B^2S/N tokens instead of BS/N , meaning multiple microbatch sequences are assigned to each GPU rather than sharding individual sequences. While this reduces all-to-all participants to N/B GPUs, it sacrifices the memory efficiency of sequence sharding—sequences are no longer split across GPUs, limiting the maximum trainable sequence length.

BASP achieves the best of both settings: like Ulysses-SP with $S_p = N$, each GPU processes only BS/N tokens, maintaining memory-efficient sequence sharding. However, like the grouped configuration, BASP limits all-to-all participants to $K = N/B$ GPUs by batching sequences within batch-topology-aware groups. This is accomplished by partitioning the B microbatch sequences into B groups of N/B GPUs each, with each group performing independent K -way all-to-all operations. The result is reduced communication overhead without compromising per-GPU memory footprint or maximum sequence length capacity.

V. EXPERIMENTAL EVALUATION

To evaluate the effectiveness of our method, we conduct experiments measuring the end-to-end execution time for representative LLM models, sequence scaling, batch scaling, and all-to-all overhead, and compare to Deepspeed-Ulysses.

Evaluation Platform: We conduct all the experiments on a GPU cluster with 2 nodes, with each node consisting of 4 NVIDIA A100 40GB GPUs connected with NVLink. All nodes are interconnected by 400Gbps InfiniBand network.

Model: To verify our claims, we perform experiments on variants of two families of models: Llama and Qwen. Table II provides the details of the model architectures.

TABLE II: Structure of language models. Llama 3.2B means Llama with 3 billion parameters, and similar for other models.

Model	Hidden Size	# Attention Heads	# Layers
Llama 3.2 1B	2048	32	16
Llama 3.2 3B	3072	32	28
Llama 3.1 8B	4096	32	32
Qwen 1.5 1.8B	2048	32	24
Qwen 2.5 3B	2560	32	32
Qwen 3 8B	4096	32	32

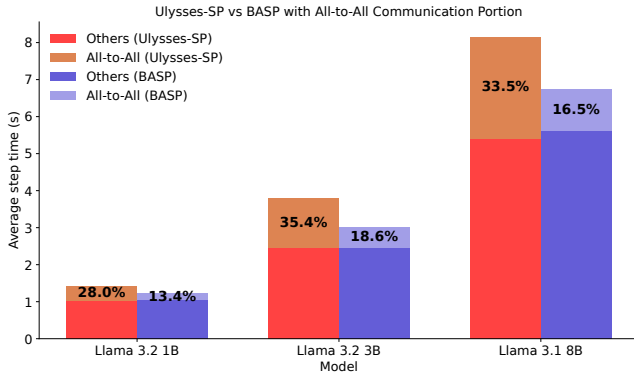
Implementation and training setup: We use Deepspeed-Ulysses as our baseline for comparison. We implement our approach by modifying the open-source DeepSpeed code. Our implementation preserves Deepspeed’s existing API while introducing batch-aware process group management and data distribution. We use Zero-3 and mixed-precision training for memory savings. The reported metrics are averaged over 30 iterations.

A. End-to-end execution time

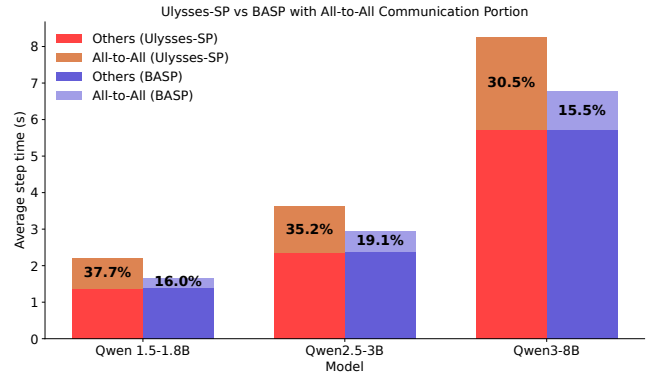
In this section, we present the end-to-end performance (average per-iteration step-time) comparison of BASP with Ulysses-SP applied to two families of models: Llama and Qwen. We train on a sequence length $16K$ with a batch size of 2 on 8 GPUs. We use micro-batch size of 2 because it is the maximum batch size that allows us to test the performance of our method against Ulysses-SP for sequence length $16K$ without OOM (Out-of-memory) issues.

Figure 7 shows that BASP consistently outperforms Ulysses-SP in all the variants of the model, achieving speedups ranging from $1.17\times$ to $1.32\times$. In the Qwen model, BASP reduces step time by 18.2–24.1% across variants of models, with the largest gain on the 1.8B model: 24% (speedup of $1.31\times$). In the Llama model, BASP reduces step time by 14.7–20.5% across variants of models, with the largest reduction on the 3.2B model (20.5%).

Communication overhead analysis: We measure the aggregate all-to-all communication time per training iteration to isolate the impact of BASP’s partitioning strategy. BASP reduces all-to-all communication time by $2.23\times$ to $3.10\times$ across all models. The Qwen 1.5-1.8B model achieves the largest reduction of all-to-all portion from 37.7% to 16% ($3.10\times$ all-to-all speedup, $1.31\times$ end-to-end speedup), while Qwen2.5-3B shows the least reduction from 35.2% to 19%. ($2.26\times$ all-to-all speedup, $1.22\times$ end-to-end speedup). These communication reductions exceed the end-to-end speedups, confirming that all-to-all operations constitute only a fraction of the total training time. For instance, in Llama 3.2-1B,



(a) Llama step time



(b) Qwen step time

Fig. 7: End-to-end step time comparison between Llama and Qwen models

despite a $2.45\times$ reduction in communication, the end-to-end speedup is $1.17\times$ because communication accounts for approximately 13.4% of the total iteration time under Ulysses-SP. The observed 2.4–3.1 $\times$ communication reduction aligns closely with our theoretical prediction of approximately $2\times$ for $K = 4$ subgroups. The communication benefit is also amplified in BASP because when $K = 4$, 4 GPUs are confined to a single node, and during the all-to-all, there is no inter-node communication.

B. Scaling with Microbatch Size

A core prediction of our method is that speedup should scale with batch size B , since larger the B , the smaller the group size $K = N/B$. We verify this by fixing sequence length at 8K and varying micro-batch size from 1 to 8 for Llama 3.2-1B. Results are shown in Figure 8.

At batch size 1, BASP and Ulysses-SP are equivalent: $K = N/B = 8/1 = 8$, so both methods perform an identical 8-way all-to-all, hence their step-time is almost identical. At $B \geq 2$, where $N \leq B$, BASP’s topology-aware grouping becomes effective, with speedup growing from $1.10\times$ to $1.26\times$ as batch size increases from 2 to 8. These results demonstrate that BASP scales favorably with increasing micro-batch size.

Communication overhead analysis: The performance benefit for BASP comes from the reduced all-to-all communication overhead as seen in Figure 8. At $B=2$, the all-to-all percent increases to 26.9% for Ulysses-SP while it decreases to 15% for BASP, and BASP achieves an end-to-end speedup of $1.1\times$. At $B = 8$, the all-to-all speedup of BASP is $85\times$ but the end-to-end speedup is $1.25\times$ compared to the Amdahl’s Law theoretical maximum of $1.5\times$ (given Ulysses-SP’s 33.8% all-to-all fraction). This denotes that as batch size increases, and the all-to-all communication becomes negligible (0.5% at $B = 8$), the performance is getting bound by the other collective operations and computation rather than all-to-all communication. Upon profiling BASP, at $B = 8$, we find that ZeRO-related communication operations become the bottleneck after sequence parallel all-to-all optimization.

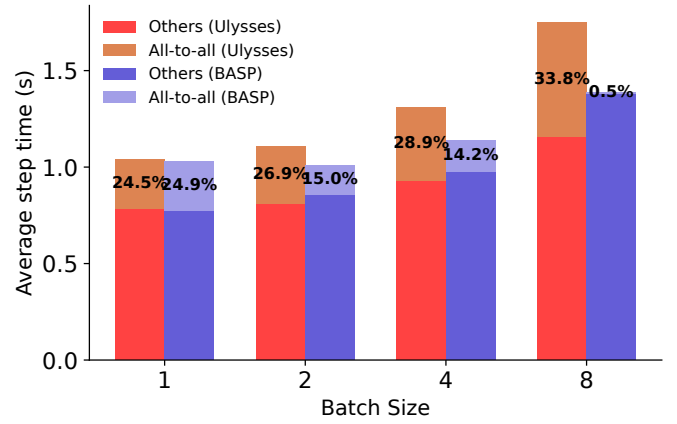


Fig. 8: Comparison of batch scaling results for our method (Llama-3.2-1B)

C. End-to-End Performance for Sequence Scaling

In the previous experiments, we showed that our method consistently performs better on the family of models. In this experiment, we show that our method scales with longer sequences. We evaluate the end-to-end time of BASP with Ulysses-SP on sequence lengths up to 32K with a batch size of 2 Llama 3.2-3B. To provide a fair comparison, we use the same setting for both methods. Figure 9 shows that BASP consistently outperforms Ulysses-SP for the sequence length that can be run with both.

Across all settings, BASP consistently outperforms Ulysses-SP, with benefits becoming more pronounced at longer sequence lengths. At short sequences (1K–4K), the improvements are modest, with BASP achieving 5.7% and 3.4% reduction in step time at 1K and 2K, respectively, and a marginal 1.8% improvement at 4K. This is expected, as communication overhead is relatively small compared to computation in smaller sequence lengths. However, as sequence length increases, BASP shows significantly stronger gains. At 8K tokens, BASP reduces step time by 13.1%, which further

increases to 18.2% at 16K and reaches 25.9% at 32K.

These results indicate that BASP provides limited but consistent gains at small sequence lengths while delivering substantial improvements in long-context regimes, where all-to-all communication dominates end-to-end training time.

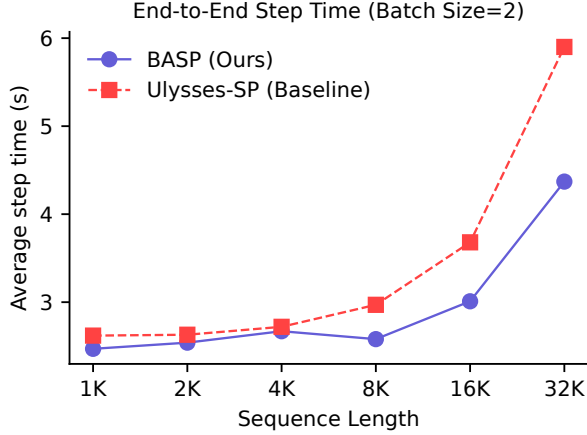


Fig. 9: Sequence scaling comparison (Llama 3.2-3B).

D. Loss Comparison Plot

To validate that our method is purely a communication optimization technique with no impact on accuracy, we compare the loss convergence of BASP and Ulysses-SP. Using the same dataset, we train Llama 3.2-3B for 800 iterations on 8 GPUs. Figure 10 shows that the loss curves for BASP and Ulysses-SP completely overlap, which validates that our method preserves the accuracy.

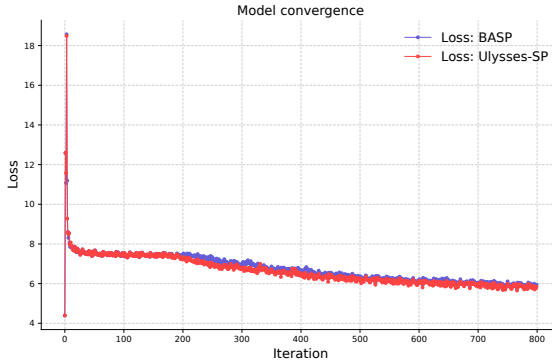


Fig. 10: Loss comparison plot showing almost complete overlap for Ulysses-SP and BASP on Llama 3.2-3B, verifying our implementation.

VI. LIMITATIONS

BASP is designed for scenarios where the number of GPUs N exceeds or equals the micro-batch size B and $N = KB$ and K is an integer. This condition holds for typical long-context

training workloads on multi-node clusters, where sequence length (and hence memory pressure) necessitates distributing each sequence across multiple GPUs. Extending BASP to support non-divisible configurations and more flexible group formation strategies is part of our future work.

VII. RELATED WORKS

There has been a surge of research on long context training in recent years as the scaling of the transformer architecture has been bottlenecked by the memory constraints. Megatron Sequence Parallelism (SP) [14] extends Megatron-LM [23], a tensor-parallel training framework, by partitioning the dropout and layernorm layers, allowing them to be distributed across multiple GPUs and therefore further reducing the memory pressure of any single GPU. Unlike Megatron-LM, which partitions attention heads across GPUs while replicating the full sequence activations on each GPU, Megatron SP additionally parallelizes along the sequence dimension and uses all-gather before attention and reduce-scatter afterward to reconstruct and redistribute the sequence partitions.

Deepspeed Ulysses [13] uses all-to-all collective among the GPUs to gather the QKV projections and performs a per-head attention computation. Unlike Megatron-SP, communication analysis shows that DeepSpeed-Ulysses keeps communication volume consistent by increasing GPUs proportional to the sequence length. However, it fails to address the all-to-all communication bottleneck for larger batch sizes.

Ring Attention [16] computes attention in a blockwise manner by leveraging the online softmax method to compute the exact attention incrementally. This approach enables training on sequences far exceeding single-device memory capacity. However, Ring Attention faces bottlenecks from peer-to-peer (P2P) communication required for transferring KV chunks, particularly when computation can no longer overlap with communication on long sequences. Striped Attention [4] improves upon Ring Attention by interleaving computation and communication more effectively, achieving better pipeline utilization through careful scheduling of block transfers.

Unified Sequence Parallelism (USP) [9] combines DeepSpeed-Ulysses and Ring Attention to counter each method's individual inefficiencies by using all-to-all collectives for short-to-medium sequences where communication can be effectively overlapped, and switching to ring-based communication for extremely long sequences where blockwise computation becomes necessary. However, Unified SP does not optimize the all-to-all collective for larger batch sizes. FlexSP [25] introduces adaptive switching between different SP strategies based on runtime profiling, dynamically selecting between Ulysses-style all-to-all, ring-based, and hybrid modes depending on sequence length, batch size, and network conditions. This adaptivity comes at the cost of runtime overhead for strategy selection and potential load imbalance during transitions.

MiCS (Mixed-Precision Communication Scheduler) [29] introduces heterogeneous sharding granularities for optimizer

states, gradients, and parameters in ZeRO [21], reducing communication volume by selectively applying mixed precision to different component types. While MiCS optimizes memory and communication for data parallelism, it does not address sequence parallelism or the topology-aware scheduling of all-to-all collectives.

VIII. CONCLUSION

We presented *Batch-Aware Sequence Parallelism (BASP)*, a simple yet effective optimization to Ulysses-SP that leverages batch structure to optimize global all-to-all communication. By decomposing a global N -way collective into independent K -way subgroup collectives, BASP reduces the number of communication phases from $(N - 1)$ to $(K - 1)$ while preserving the per-GPU memory footprint and sequence partition size. Experiments on Llama and Qwen models show that BASP achieves a speedup of $1.21\times$ on Llama 3.1-8B and $1.32\times$ on Qwen 1.5-1.8B compared to Ulysses-SP with a reduction of all-to-all time by up to 67.7% in Qwen 1.5-1.8B.

ACKNOWLEDGMENT

This material is based upon work supported by the United States Department of Energy under grant DE-SC0026344 and National Science Foundation under Grant No. SHF-1943114. Clemson University is acknowledged for generous allotment of compute time on the Palmetto cluster.

REFERENCES

- [1] Anthropic. *Claude Opus 4.7*. Anthropic, 2026. Accessed: May 1, 2026.
- [2] Daiyaan Arfeen, Zhen Zhang, Xinwei Fu, Gregory Ganger, and Yida Wang. Pipefill: Using gpus during bubbles in pipeline-parallel llm training. *Proceedings of Machine Learning and Systems*, 7, 2025.
- [3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [4] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers. *arXiv preprint arXiv:2311.09431*, 2023.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] Qiaoling Chen, Qinghao Hu, Guoteng Wang, Yingdong Xiong, Ting Huang, Xun Chen, Yang Gao, Hang Yan, Yonggang Wen, Tianwei Zhang, et al. Lins: Reducing communication overhead of zero for efficient llm training. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, pages 1–10. IEEE, 2024.
- [7] Robert Chew, John Bollenbacher, Michael Wenger, Jessica Speer, and Annice Kim. Llm-assisted content analysis: Using large language models to support deductive coding. *arXiv preprint arXiv:2306.14924*, 2023.
- [8] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- [9] Jiarui Fang and Shangchun Zhao. Usp: A unified sequence parallelism approach for long context generative ai. *arXiv preprint arXiv:2405.07719*, 2024.
- [10] Hao Ge, Junda Feng, Qi Huang, Fangcheng Fu, Xiaonan Nie, Lei Zuo, Haibin Lin, Bin Cui, and Xin Liu. Bytescale: Communication-efficient scaling of llm training with a 2048k context length on 16384 gpus. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 963–978, 2025.
- [11] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.
- [12] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [13] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deep-speed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.
- [14] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeibi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models, 2022. URL <https://arxiv.org/abs/2205.05198>.
- [15] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence parallelism: Long sequence training from system perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2391–2404, 2023.
- [16] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.
- [17] Man Liu, Xingchen Liu, Xingjian Tian, Bing Lu, Shengkay Lyu, Shengquan Yin, Wenjing Huang, Zheng Wei, Hairui Zhao, Guangming Tan, et al. Taco: Efficient communication compression of intermediate tensors for scalable tensor-parallel llm training. *arXiv preprint arXiv:2604.24088*, 2026.
- [18] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- [19] Zhangqiang Ming, Yuchong Hu, Wenxiang Zhou, Xinjue Zheng, Chenxuan Yao, and Dan Feng. Adtopk: all-dimension top-k compression for high-performance data-parallel dnn training. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, pages 135–147, 2024.
- [20] Open AI. *ChatGPT*. Open AI, 2026. Accessed: April 13, 2026.
- [21] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: international conference for high performance computing, networking, storage and analysis*, pages 1–16. IEEE, 2020.
- [22] Stefano Rando, Luca Romani, Alessio Sampieri, Luca Franco, John Yang, Yuta Kyuragi, Fabio Galasso, and Tatsunori Hashimoto. Long-codebench: Evaluating coding llms at 1m context windows, 2025. URL <https://arxiv.org/abs/2505.07897>.
- [23] Mohammad Shoeibi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [25] Yujie Wang, Shiju Wang, Shenhan Zhu, Fangcheng Fu, Xinyi Liu, Xuefeng Xiao, Huixia Li, Jiashi Li, Faming Wu, and Bin Cui. Flexsp: Accelerating large language model training via flexible sequence parallelism. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 421–436, 2025.
- [26] Jinghan Yao, Sam A Jacobs, Masahiro Tanaka, Olatunji Ruwase, Hari Subramoni, and Dhabaleswar Panda. Training ultra long context language model with fully pipelined distributed transformer. *Proceedings of Machine Learning and Systems*, 7, 2025.
- [27] Geng Zhang, Shenggan Cheng, Xuanlei Zhao, Ziming Liu, and Yang You. Helixpipe: Efficient distributed training of long sequence transformers with attention parallel pipeline parallelism. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 413–424, 2026.
- [28] Hang Zhang, Xin Li, and Lidong Bing. Video-llama: An instruction-tuned audio-visual language model for video understanding. In *Proceed-*

ings of the 2023 conference on empirical methods in natural language processing: system demonstrations, pages 543–553, 2023.

- [29] Zhen Zhang, Shuai Zheng, Yida Wang, Justin Chiu, George Karypis, Trishul Chilimbi, Mu Li, and Xin Jin. Mics: near-linear scaling for training gigantic model on public cloud. *arXiv preprint arXiv:2205.00119*, 2022.
- [30] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.